SRDT MULTIMODAL AI CHATBOT

PROJECT REPORT & VERIFICATION SUMMARY

16	14	2
Recorded checks	Working	Degraded

Prepared from the supplied project repository and its included automation test record. This report distinguishes recorded results from checks independently performed during packaging.

Project: SRDT-MM-Chatbot
Package: source code, setup guide, verification notes and this report

1. Executive Summary

The SRDT Multimodal AI Chatbot is a full-stack prototype built with a Streamlit frontend and FastAPI backend. It brings together conversational AI, document-grounded question answering, image understanding, audio output, image generation/enhancement, web search, and model-provider fallback.

2. Objectives

• Provide a single interface for conversational AI.

• Accept images and supported documents as inputs.

• Retrieve relevant document passages to ground answers (RAG).

• Route requests across configured AI providers with fallback behavior.

• Offer speech synthesis and image generation/enhancement utilities.

• Support offline demonstrations through a deterministic mock provider.

3. Technology Stack

Layer	Technology / role
Frontend	Streamlit interface and HTTP API client
Backend	FastAPI, Uvicorn, Pydantic
Database	SQLAlchemy; SQLite default; PostgreSQL/pgvector option
AI providers	Google Gemini, xAI Grok, optional Groq, offline mock
Retrieval	Document extraction, chunking, embeddings and cosine similarity
Media	Vision, gTTS speech synthesis, image generation and image processing
Testing	pytest suite and included automation test harness

4. System Architecture

The user interacts with the Streamlit frontend. The frontend sends requests to FastAPI routers for chat, model discovery, document operations and multimodal features. Service modules isolate provider integrations, retrieval, parsing, web search, audio, vision and image processing. SQLAlchemy persists application data; SQLite is the local default, with PostgreSQL/pgvector available for deployments that configure it.

Component	Responsibility
frontend/app.py	User interface, model selection, chat and upload interactions
backend/app/api/	HTTP endpoints for chat, models, documents and multimodal actions
backend/app/services/provider_router.py	Provider selection and fallback attempts
document_service.py + rag_service.py	Document extraction/chunking and grounded retrieval
backend/app/db/	Database sessions and vector adapter
backend/tests/	Automated test cases

5. Main Features

• Text chat with session persistence and model selection.

- Gemini/Grok/Groq integrations with fallback and offline mock mode.
- PDF, DOCX, TXT and Markdown document ingestion for RAG.
- Image understanding and multimodal endpoints.
- Text-to-speech output.
- Text-to-image generation with optional resizing/sharpening enhancement.
- DuckDuckGo web-search grounding when the external service responds.
- Document listing, deletion and generated-document versioning.

6. Recorded Functional Verification

The included automation_test_report.json contains 16 checks. The statuses below are transcribed from that file; they are not claims that every external service will remain available.

Recorded check group	Status	Recorded observation
Health and database	WORKING	SQLite healthy; vector retrieval available
Model registry	WORKING	Grok, Gemini, Groq and mock entries registered
Chat persistence	WORKING	Chat session and messages persisted
Provider failover	WORKING	Fallback provider selected after preferred provider failure
Document parsing / chunking	WORKING	TXT parsed and chunk stored
Semantic RAG	WORKING	Relevant context chunk and citation returned
DuckDuckGo search	DEGRADED	Zero results or rate limiting; graceful fallback
Vision mock flow	WORKING	Image input accepted by mock analysis
Text-to-speech	WORKING	MP3 stream generated
Image generation	WORKING	Image bytes generated in recorded run
Document versioning	WORKING	Version advanced from 1 to 2
Emotional support flow	WORKING	Supportive response generated
Crisis guardrail	WORKING	Guardrail intercepted test input
Gemini cloud API	WORKING	Recorded readiness response received
xAI Grok cloud API	DEGRADED	HTTP 403 due to quota/account balance
Frontend UI checks	WORKING	Model switcher, uploader, action chips and audio present

7. Independent Checks During Packaging

Check	Result	Meaning
Python source compilation	SUCCESS	All discovered Python source files compiled successfully.
pytest collection	BLOCKED	The environment lacked duckduckgo_search; import failed while loading backend tests.

No independent pytest pass count is claimed. Install the dependencies listed in backend/requirements.txt and frontend/requirements.txt, then run python -m pytest -q in a configured environment.

8. Known Limitations & Recommendations

• Web search and cloud model availability depend on third-party services, quotas and network conditions.

• Configure valid API keys and verify provider quota before a live demo.

• Add CI to install dependencies and run tests in a clean environment.

• Before production deployment, add authentication, rate limiting, upload limits, structured logging, secret management and restrictive CORS settings.

• Image resizing/sharpening can improve perceived clarity but is not equivalent to learned super-resolution and cannot guarantee recovery of missing detail.

• Emotional-support functionality is not medical care or crisis counselling. In India, emergency support is available through 112; Tele-MANAS can be reached at 14416.

9. Setup & Run

Install (Windows PowerShell, project root):
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r backend/requirements.txt -r frontend/requirements.txt
Copy-Item .env.example .env

Terminal 1 — backend:
python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000 --reload

Terminal 2 — frontend:
python -m streamlit run frontend/app.py

Open http://localhost:8501. API docs: http://127.0.0.1:8000/docs. Health: http://127.0.0.1:8000/health.

10. Conclusion

The repository provides a modular MVP foundation for a multimodal AI assistant. The supplied automation record reports 14 working checks and 2 degraded external-service checks. Source compilation succeeded during packaging, while an independent pytest run was blocked by a missing dependency. The project is ready for further local setup, configuration and validation.